

SURGE099

Voice Controlled Room Lighting

Alexa Equivalent for Smart Home Control

Abhishek Mehta

2024EE30767

15-Day Progress Report



PHASE 01

Learning Phase

Hardware Fundamentals & Arduino mini projects



Traffic Light System

Multi-LED state machine with timed transitions. Applied digital I/O pin control and timing logic.



Reaction Speed Game

User-input triggered measurement system with Buzzer feedback. Practiced interrupt handling and timing.



LED Patterns & Sensors

Blinking sequences, button/sensor interfacing. Built foundational understanding of digital circuits.

PHASE 02

Platform & Component Evaluation

ESP32 vs Raspberry Pi · Audio Components · Sensors

Criteria	ESP32	Raspberry Pi
Power Consumption	Very Low (240 mA)	High (700 mA+)
Wi-Fi / Bluetooth	Built-in	Built-in
Boot Time	< 1 second	~30 seconds
Complexity (for IoT)	Lightweight, ideal	Overkill
Cost	Low (~\$5)	Moderate (~\$35)

✓ Final Decision

ESP32 selected as the primary microcontroller for its low power draw, native Wi-Fi, and embedded suitability.



Audio Research

- INMP441 I2S digital mic (selected)
- Wireless mic amplifier options evaluated
- Analog vs digital mic noise analysis

PHASE 03

Voice Processing Architecture

Two candidate approaches evaluated for keyword recognition accuracy

Option A — Local TinyML

- + No internet dependency
- + Very low latency
- Limited to fixed keywords only
- Needs large labelled dataset
- Accuracy drops in noisy environment
- Model training complexity on ESP32

Accuracy: Moderate

VS

Option B — Cloud NLP

- + Full natural language understanding
- + High accuracy (Wit.ai / Whisper API)
- + No on-device training needed
- + Extendable to complex commands
- Requires stable Wi-Fi
- Small round-trip latency

Accuracy: High ✓ Selected

PHASE 04

System Architecture & Next Steps



End-to-End Voice Control Pipeline

Next Steps

1. Implement ESP32 I2S audio capture firmware
2. Integrate Wit.ai API for intent recognition
3. Build relay-based light toggle circuit
4. End-to-end integration & latency testing
5. Noise resilience testing and optimisation
6. Final demo and documentation